Validation Specification (Update)

Face Recognition-Based Smart Attendance Management System

Project ID: 20

Version 1.0

UE24CS341A - Software Engineering, Jackfruit Mini-Project

Team Details

Field	Details
Team Name	SE GROUP
Team Member 1	MITHUN S | SRN: PES2UG24AM208
Team Member 2	BHARATH H G | SRN: PES2UG24AM197
Team Member 3	VARUN C H | SRN: PES2UG24AM180
Team Member 4	YASHWANT ADITYA VARMA | SRN: PES2UG24AM190
Prepared by	MITHUN S
Date Created	5.10.2026

1. Purpose

This document updates the project's Validation Spec now that the Architecture and Software Design documents exist. Per the course guidelines, this stage is focused on planning only: it defines how each requirement will be validated (which test level, which design element is responsible) and lays out the functional, security, and non-functional validation approach. The actual test cases with pass/fail results are produced later, in the Phase-3 Test Plan Document, using this plan as its starting point.

2. Validation Approach Overview

Given the team size (3-4 students) and project scope, validation will be carried out through a mix of:

- Unit-level checks: calling individual service functions directly (e.g. compute_percentages(), get_face_detector()) with controlled inputs to confirm correct return values and error handling.
- Integration checks: exercising a full Flask route end-to-end (e.g. POST /enroll, then GET /dashboard) using Flask's test client, to confirm the layers work together correctly.
- Manual/system checks: hands-on testing of the camera-dependent flows (enrollment capture, live recognition) which cannot be fully automated without a physical webcam and real faces, carried out by team members acting as test subjects.

Testability note: all requirements in the SRS (Section 5) are written as observable, checkable outcomes (e.g. 'the system shall mark present/absent', 'the system shall reject an image with zero or multiple faces') specifically so each one maps to at least one concrete test case below, satisfying the SRS quality bar of being verifiable (see SRS Section 6.4).

3. Requirements Validation Mapping

Req. ID	Validation method	Responsible module
REQ-EN-1	Manual/system: enroll a student, confirm 3+ images saved under dataset/	FaceEnrollmentService
REQ-EN-2	Manual: attempt capture with 0 and 2+ faces in frame, confirm rejection/recapture prompt	FaceEnrollmentService
REQ-EN-3	Unit: inspect trainer/trainer.yml exists and loads after enrollment; confirm no raw image path stored in DB	FaceEnrollmentService
REQ-EN-4	Manual: re-enroll the same student, confirm old images are supplemented/model retrained	FaceEnrollmentService
REQ-AT-1..3	Manual/system: enroll 2+ people, run attendance capture, confirm correct Present/Absent split	FaceRecognitionService
REQ-AT-4	Integration: query the attendance table after a session, confirm date/section/timestamp recorded	AttendanceService
REQ-AT-5	Manual: present an unenrolled face, confirm it is not force-matched to an enrolled student	FaceRecognitionService
REQ-OV-1..2	Integration: call the override path with a changed status, confirm the stored record updates and is logged	AttendanceService
REQ-OV-3	Integration: attempt override as a Student-role session, confirm rejection	AuthRoleManager
REQ-AN-1..3	Unit: seed known attendance rows, confirm compute_percentages() returns the expected %, and sub-75% rows are flagged	AnalyticsService
REQ-AN-4	Integration: load /dashboard as a Student-role session, confirm only their own record is visible	AuthRoleManager + AnalyticsService
REQ-RP-1..3	Integration: call /export, confirm CSV headers, row content, and filename match spec	ReportService
REQ-AU-1..4	Integration: attempt login with wrong password, confirm rejection; inspect DB to confirm hashed (not plaintext) passwords	AuthRoleManager

4. Functional Validation Plan

- Happy path: enroll 2-3 students, take attendance with all of them present, confirm each is marked and appears correctly on the dashboard.
- Edge case - duplicate roll number: attempt to enroll two students with the same roll number, confirm the second is rejected with a clear message.
- Edge case - poor lighting / partial face: attempt enrollment capture in low light, confirm the system does not silently save unusable samples.
- Edge case - no camera attached: disconnect/disable the webcam, confirm a clear error message rather than a crash.
- Edge case - zero sessions recorded: load /dashboard before any attendance has been taken, confirm it shows 0% / an empty state rather than dividing by zero.

5. Non-Functional Validation Plan

5.1 Performance

Manually time a full attendance capture for a section of 5-10 enrolled students; confirm recognition of all visible faces in a single captured frame completes within the 5-second target in SRS Section 6.1 (note: the current implementation processes a continuous camera stream rather than one still frame, so this will be measured as per-frame processing latency).

5.2 Security

See the pen-testing checklist in Section 6 below; this is the 'security validation' item required by the course guidelines.

5.3 Usability

Have one team member unfamiliar with the code attempt the full enroll -> take attendance -> dashboard flow using only the UI, timing how long it takes, against the SRS Section 6.4 target of under 1 minute for a class of 60 (scaled down proportionally for the smaller demo class size).

6. Security Validation / Light Penetration-Testing Plan

Per the course requirement that security validation and pen testing be at least planned at this stage, the following basic checks apply now that Auth/Role Manager is implemented (several are already executed and documented as passing in the Phase-3 Test Plan; the remainder are listed here for completeness):

- Attempt to access a protected route (e.g. /dashboard, /export) directly by URL without logging in; expect a redirect to login, not the page content.
- Attempt login with a SQL-injection-style username (e.g. " ' OR 1=1 -- "); since the implementation uses parameterized queries (sqlite3 placeholders), expect this to simply fail as an invalid login, not bypass authentication.
- Inspect the database file directly (e.g. with a SQLite browser) to confirm the students/users table stores password hashes, never plaintext passwords (REQ-AU-2).
- Confirm, by inspecting the repository, that no raw face images or the trained model file are committed to GitHub (both should be excluded via .gitignore) – a privacy/security check specific to this project's biometric data.
- Confirm session cookies expire after the configured inactivity window (SRS Section 6.3) by waiting past that window and retrying a protected action.

7. Requirements Traceability

This document's Section 3 mapping feeds directly into the Requirement Traceability Matrix in SRS Appendix C: the 'Architecture Ref.' and 'Design Ref.' columns there should be filled in using the Architecture Document Section 4 and this SW Design Document's Section 2/6/7, and the 'Test Case ID' columns will be completed once the Phase-3 Test Plan assigns concrete IDs (e.g. UT-xx, IT-xx, ST-xx) to each row above.